

Homework #0: Python and Data Structures Warm Up

Objective

The main goal of this homework assignment is to help you practice and remember basic data structure algorithms using python. We will be implementing similar algorithms later in this course. A secondary goal is to refresh general principles of Python programming including object-oriented programming and recursion, which will also be needed on future assignments.

Summary

In this assignment, you are tasked with implementing a simple arithmetic expression calculator using python. The input math expressions will be provided as JSON files. Each file will define an Operator Tree with the entire hierarchy of operations to execute. Your goal is to read these files, create a tree data structure that represents the Operator Tree for further evaluation. You will also have to use post-order tree traversal to create a representation of the mathematical expression in post-fix notation which can be easily evaluated using a classic stack-based algorithm.

Problem Description

You are given a set of files in JSON format. Each file represents a mathematical expression. In this case, the JSON format was picked since it is a commonly used format to represent hierarchical data such as trees. Mathematical expressions are trees after all, where the bottom nodes represent operands and intermediate nodes follow the order of operations, with the root being the last operation that needs to be performed. Figure 1 shows a basic example for the expression: $(6 * 8) * 10$.

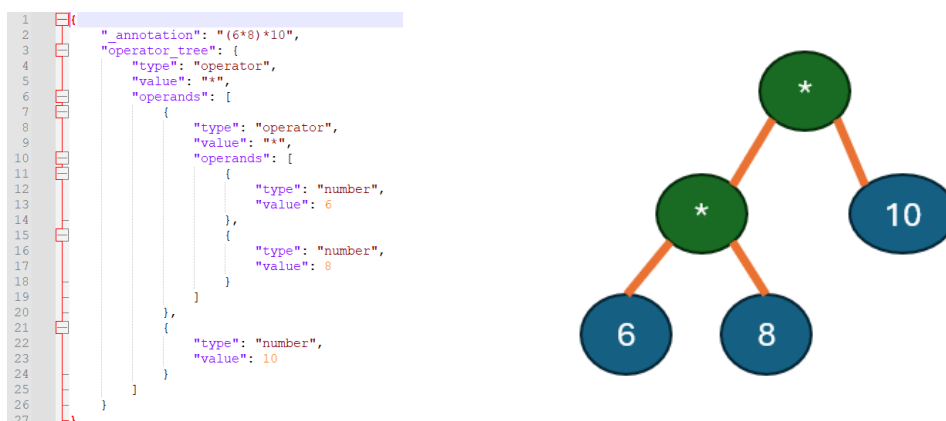


Figure 1. Example of Mathematical Expression. Left side is JSON representation. Right side is its corresponding Operator Tree representation.

Your goal will be to read the JSON file (easy to do with python libraries) and then analyze the content of the file to create an object-oriented representation of the mathematical expression in memory. You will evaluate the expression using two different methods, one that uses the tree structure directly, and another that transforms the expression first and then uses stacks. To make things simple, the whole process has been divided into smaller steps which you should complete sequentially:

Step 1. Loading the JSON file.

This step is relatively easy in Python. However, you will notice that you are required to handle command line arguments. If you are unfamiliar with these, you will have to check the documentation for more details. The idea is that the user should be able to run your program using commands like:

```
python calculator.py math_expr_4.json
```

This should execute the calculator program, loading the required data from the file “math_expr_4.json”. you will need the built-in python libraries: json and sys.

Step 2. Creating the Operator Tree structure using the data loaded from JSON.

This step requires converting the JSON data in memory into a proper OperatorTree structure. The input follows the format shown in the left panel of Figure 1. The root node of the JSON data will always contain a field named “operator_tree”. This field provides the recursive definition of the tree. Every node is represented as a Python dictionary, where the field “type” defines the class that should be used to load the node. In this case, nodes with type=“operator” should be loaded as instances of the Operator class, and nodes with type=“number” should be loaded as instances of the Operand class. For operators, “value” represents the actual operator, which can only be one out of four: “+” for sum, “-” for subtraction, “*” for multiplication and “/” for division. For operands, “value” is the actual number for this operand. On the right panel of Figure 1, the green nodes represent instances of the Operator class, and the blue ones are instances of the Operand class.

As a requirement, you must do this process using a call to the function “BuildFromJSON” from the OperatorTree class. This function receives the raw JSON data as loaded from the file, and it starts the recursive process of constructing the tree. Note that the root node itself might be either an operator or an operand, and OperatorTree.BuildFromJSON should handle both cases. For operators, these should be constructed using their own function BuildFromJSON, which gets a JSON node as input, and produces an Operator object as output. By definition, ^{operators(*) are never leaf nodes} all operators are internal nodes of the operator tree which means that they will have children which are stored as a list in the field “operands” of the corresponding JSON node (see Figure 1). In turn, each child could be either another operator (an internal node), or an operand (a leaf), and the function BuildFromJSON from the corresponding class should be called. Note that Operands represent the base case which stops the recursion. ^{is this always true?}

Step 3. Use the Operator Tree to evaluate the Mathematical Expression.

This step requires implementing the “evaluate” function on the OperatorTree, Operator and Operand classes. For the OperatorTree class, this simply requires calling the “evaluate” function of the root node. For the Operand class, “evaluate” requires simply returning the value stored in that node. Meanwhile, for

^{just return getvalue()}

the `Operator` class, “evaluate” implements the main logic of the process of evaluating the mathematical expression. It must apply the corresponding operator to the results from recursively evaluating its children.

The original JSON file includes the field “_annotation” which shows the original mathematical expression in human-readable format. This expression can be used to calculate the expected value for any given test case. Any discrepancies can be used to find and debug errors.

Step 4. Generate a List of Values in Post-Order.

Post order traversal of the tree

This step requires implementing the “post_order_list” function of the `OperatorTree`, `Operator` and `Operand` classes. For the `OperatorTree` class, this function must create an empty list which will be passed to the recursive calls to this function on the tree nodes. Operands are the base case, and they simply add themselves to the list before returning. Operators also add themselves to the list (in post order), but they must call the function recursively on their children. Printing the resulting post-order list should generate the equivalent of the same mathematical expression in post-fix notation.

Step 5. Use the post-order list to evaluate the Mathematical Expression.

This step takes the post-order list and applies a simple algorithm that can evaluate mathematical expressions in post-fix notation using a stack. The idea is that the elements on the post-fix notation are read sequentially left-to-right. Every time an operand (e.g., a number) is found, this value is simply pushed into a stack. Then, every time an operator (e.g., “+”, “-”, etc.) is found, the last two values on the stack are popped, and the operator is applied to these two values, and then the result is pushed into the stack. A few things to notice:

- The stack should have exactly one element at the end. If that is not the case, then there is an error somewhere. This value should be returned as the result of the evaluation.
- Operands are pushed in a certain order, but stacks will pop them in LIFO order. You need to mind this since some operators are NOT commutative.

The output produced by this method should be identical to the result produced in Step 3. Any differences will indicate that there is an error somewhere in the code.

Implementation Details

For this work, you will be given a complete project in Python with multiple classes for specific tasks. To keep things simple, you are given some **code skeleton**. This includes the classes and functions to be implemented. Your main task is to implement specific functions. **Table 1** summarizes the code and data files given to you. The details of the classes to implement are below:

OperatorTree. Main class used to represent a **mathematical expression**. It only has one attribute, “root”, which represents the first **OperatorTreeElement** at the root of the whole tree. It provides methods for **creating**, **traversing** and **evaluating** the tree.

OperatorTreeElement. Abstract class used to represent nodes in the `OperatorTree` class. A node here can be either an **operator** or an **operand**. The abstract class only defines one generic attribute, “_value”, which represents the content of the node. This attribute is inherited by the children. The class defines two concrete methods, “get_value” and “__repr__” to read and print its current value. It also defines

abstract/virtual methods for **creating**, **traversing** and **evaluating** the tree, which are meant to be implemented by its child classes.

Operator. This is the class that does most of the heavy work here. It represents operators ("+", "*", "/" and "-") on the operator tree. It has a list of children representing the operands over which the operator will be applied. Notice that some of these children can be operators themselves, leading to a recursive definition of the mathematical expression. For the sake of simplicity, we will assume that our class only needs to worry about binary operations, and therefore it will always have exactly two children. Feel free to extend the class to support non-binary relations like adding or multiplying more than 2 values using a single operator. In terms of methods, the class inherits and overrides the required methods for **creating**, **traversing** and **evaluating** the tree.

Operand. This class simply holds the values of the operands that are located at the leaves of the operator tree. It still must provide very basic implementation for the base cases of the recursions of the operations for **creating**, **traversing** and **evaluating** the tree.

Name	Type	Description	Modify
calculator.py	Code	Entry point for the program. It puts everything together.	Yes
CSC480_Calculator/___init___py	Code	Python module file. It allows to register the classes and functions that are part of the module.	No
CSC480_Calculator/operator_tree_element.py.py	Code	Defines the OperatorTreeElement class.	No
CSC480_Calculator/operand.py	Code	Defines the Operand class.	Yes
CSC480_Calculator/operator.py	Code	Defines the Operator class.	Yes
CSC480_Calculator/Operator_tree.py	Code	Defines the Operator Tree class.	Yes
math_expr_1.json, math_expr_2.json ...	Data	Each file contains a mathematical expression meant to be used as a test case for your code.	No

Table 1. Summary of files included in the code project.

Additional Considerations

- 1) Base code is provided for this work. You must respect this code. Not using the functions and classes provided can lead to **severe scoring penalties**.
- 2) Programs that do **not compile will receive a score of 0. No exceptions**.

Delivery Instructions

You are being provided with a Zip file including a basic PyCharm project. You are required to implement your solution using the code provided. Once done, **you should simply zip back the entire project and submit it as a ZIP file**.

1. **DO NOT rename any internal project files!**
2. **Using other compression formats (.rar) will lead to a penalty.**
3. **If you are using a virtual environment in python, DO NOT include it on your zip file! It will make the project huge!**

File names. The zip file that you submit should use “[Last Name(s)], [Given Name(s)].zip” as it appears in D2L. For example, “Kenny Davila Castellanos” (Davila Castellanos is two last names), would have to submit the homework with the name “Davila Castellanos, Kenny.zip”. Another student named “Kenny Mauricio Davila” (Mauricio is a middle name), would have to submit the homework as “Davila, Kenny Mauricio.zip”.

Policies

1. This homework is meant to be worked **individually**.
2. You must follow the delivery instructions.
3. All general policies about Plagiarism and Cheating apply to this homework. If you plagiarize or receive code from other people, you will be caught and you will receive a score of 0, and the academic integrity violation will be filed.
4. You must not use Chat-GPT or any other code generators to directly create code for this assignment. Doing so will be considered a form of cheating and are also eligible to receive a score of 0.
5. Do not post your solutions online and do not share them with anyone. It is your responsibility to safe guard your private data.
6. Code that does not compile due to syntax and/or semantic errors will automatically receive a score of 0. It is hard to assign partial credit when I cannot even run your code.
7. Must use PyCharm if you want to receive technical support from my end.
8. Late submissions without justification might receive penalties proportional to the number of hours late. See the syllabus for concrete details on late penalties.
9. The code and documentation highlight using specific approaches for certain things. **You are expected to follow these approaches. Any different approach that you use without consulting me first is likely to receive a score of 0.**
10. Do ask for help if anything is unclear, but do it in a timely manner (e.g., by e-mail or during the Office Hours).